

CS432/532 Computer and Network Security

Spring 2026 - Term Project

Secure Channel Application

Deadline: May 8, 2026, Friday, 22:00 (Be aware that the deadline is not midnight)

Demos: Week of May 11 (scheduled to be announced later)

Second chance demos: Time and schedule will be announced later

You are not allowed to give any part of this document to any online or offline tool other than PDF reader.

We will be using several AI tool responses as benchmarks for the plagiarism testers and any above-threshold similarity will be treated as plag.

General Rules

- **CS532 students** should complete the project alone. CS532 students have an option to propose their own projects instead of doing this one; however, in such a case, the project proposal must be sent to the instructor before April 24, 2026. If no proposals are received, we will assume that you will do this project.
- **CS432 students:** You can work in groups of two people. If you cannot find a group mate, you can do the entire project alone. We will not group people and will not facilitate groupings. Your TA Ecem Şamlıoğlu will collect group information (please wait for her announcement).

Equal distribution of the work among the group members is essential. Both members of the group should submit all the codes for both the client and the server (no email submissions). Both members should be present during the demos. In case of any dispute within the group, please do not allow the problematic group member to submit the code, so that they will not be graded. However, if a group member submits the same code, then the other member automatically accepts their contribution.

Programming Rules

- You must use Python in your application.
- Your application should have a graphical user interface (GUI). **It is not a console application!**
- Your code should be clearly commented. This may affect up to 5% of your grade.
- You must use **TCP sockets** as mentioned in the socket lab session. You are not allowed to use any other type of socket. Especially, please do **not** write to us to ask for permission to use Websocket or any API for communication.
- Client and server programs should be working when they are run on at least two separate computers. So please test your programs accordingly.

- Your program should be portable. It should not require any dependencies specific to your computer. We will download, compile and run it. If it does not run, it means that your program is not running. So do test your program before submission.

Submission, Demos and Oral Examination

- Submit your work to SUCourse. **Both group members must submit.**
- Server and Client codes must be in separate files. Please submit these two .py files without zipping them.
- All code files names must start with **XXXX_Surname_OtherNames**, where XXXX is your SUNet username (e.g. alaaalmouradi_Almouradi_Alaa). Put the words `_server`, `_client` after this fixed prefix. Of course there should be an extension for the files (i.e. .py).
- You will be invited for a demonstration of your work. During the demos, you will be deeply questioned about your code and any indication of using AI-generated code will result in immediate zero and this time since this is the project, will be reported for disciplinary investigation. Date and other details about the demo will be announced later.
- The submitted code will be used during the demo. No modification on the submitted code will be allowed.
- No late submissions.
- In case there is an unclear part in the project, you can use the office hours or send email to course TAs.

Second chance demo rules

After the announcement of the project grades, you will have three days to correct the problematic parts to make a second demo. However, you will be able to claim only half of the deducted points in the second demo. No group changes are possible during the second demo.

Second chance demo is only for the groups who did regular submission and demo; we do **not** allow using this right for unofficial extension for the entire project with 50 points penalty.

Please check out the syllabus of the course for plagiarism, AI policy and other general rules.

Good luck!

Albert Levi

Alaa Almouradi – alaaalmouradi@sabanciuniv.edu

Saleh Alshurafa- saleh@sabanciuniv.edu

Elif Ecem Şamlıoğlu - esamlioglu@sabanciuniv.edu

Introduction

In this project, you will implement a client-server application for establishing a *secure* and *authenticated* message broadcast mechanism among subscribers of different channels (in total there will be three channels). In the application that you will implement, there will be *Client* and *Server* modules.

The *Server* module:

- receives the messages from clients and broadcasts them to the subscribed clients on a specific channel.

The *Client* module:

- registers to the Server and subscribes to a channel,
- gets authenticated to the Server,
- connects to the Server to send and receive messages in a secure and authenticated way through the channel subscribed.

You should design and implement a user-friendly and functional graphical user interface (GUI) for client and server programs. All activities and data generated by the server and the clients should be reported in text widgets at their GUIs. These include (but not limited to):

- RSA public and private keys in hexadecimal format,
- AES keys and IVs in hexadecimal format,
- Random challenges and responses calculated for the challenge-response protocol runs in hexadecimal format,
- Digital signatures in hexadecimal format,
- List of online clients (on the server side),
- Verification results (digital signatures, HMAC verifications, etc.),
- Message transfer operations and message texts,
- HMAC values, session keys, etc. in hexadecimal format,
- And all other details that are needed to follow the flow of execution. We cannot explain all the details of the things that need to be showed on the GUI within the project document; thus, you may take this as the golden rule: "**if we cannot follow what is going on, we cannot grade**". And please do not ask us if you should display this or that; please use your own reasoning thinking of the golden rule above. Another related rule is that we will not follow the execution of your code in debug console; we will grade using whatever you put on the GUIs.

Enrollment and User Authentication

Firstly, the client performs *enrollment* operation. You will implement an enrollment protocol between client and the server. The initial contact of each client to the server is the *enrollment* process. Only the enrolled users (clients) can be authenticated by the server; this authentication is done via a challenge-response protocol. Enrollment and authentication processes are explained in the following subsections.

We will be providing you with two RSA-3072 public-private key pairs in the project pack. These key pairs will be used by the server; one for encryption/decryption and one for signing/verification purposes. You have to read the RSA keys on server start-up.

Throughout this project, SHA3-512 is the only hash algorithm you will use. This applies to all hashing operations including password hashing, RSA signature and verification, and HMAC computations.

Enrollment Phase

Before doing anything, a client has to enroll to server in a secure way if he/she has not done it yet. Before making the connection to the server, the client program should load the server's public keys (one for encryption and the other for signature verification) from the file system (these keys are provided in the project pack).

Then, the user enters the IP address and the port number of the server, as well as his/her selected username, selected password and selected channel to get subscribed from the client GUI. Note that there will be only three channels that can be subscribed (via selecting from client GUI): IF100, MATH101 and SPS101 ☺. Channel subscription will be completed in enrollment phase and cannot be changed later.

The client should connect to the server using the IP address and the port number entered. If the connection is not successful, it should show an error message and ask the user to enter the port number and IP address information again. If the connection is successful, then client takes the hash of the entered password as well as the hash of the reversed password string using SHA3-512. After that, it concatenates these hashes along with the username and the selected channel name. Then, it encrypts this concatenated message by using the RSA public key (the one for encryption/decryption) of the server and sends the encrypted message to the server.

Server receives this encrypted message and decrypts it using the corresponding RSA private key. Then, it parses the decrypted message and gets the username, the hash of the password that the user entered and the hash of the reversed password, and the selected channel name. After that, Server should check if there is another enrolled client with the same username. If so, it should send back an error message to the client stating that the username has already been taken. Otherwise (if the username is unique amongst enrolled clients), it should save the username, the hash of its password, the hash of its reversed password and subscribed channel name to its database (it is up to you how to implement this), and send a message to the client stating that the enrollment is successful. The success/error messages sent by the server must be signed by server's signature RSA key.

Once the client receives signed response from the server, it first verifies the server's signature using the corresponding public key. If the response is verified, client checks whether the response is "error" or "success". If "error" is received, then the client can try again with another username. If "success" is received, that means the client has been successfully enrolled to the system.

After successful enrollment, client should be able to login to the system with the username and the password it selected. Please note that, the clients do not save or store their passwords and the user should always enter it from scratch when the password is asked.

Enrollments are permanent, meaning that after any type of disconnections (Server or clients), the enrolled users must be remembered.

Authentication (Secure Login) Phase

This phase is the implementation of a secure login protocol for the client to get authenticated to the server. In this phase, a challenge-response protocol is run between the client and the server.

If not already entered, first the user should enter the IP address of the server and the port number. Then, the client connects to the server and sends an authentication request to it, together with the username (not the password!). This request is in clear text and initiates the challenge-response protocol. After that, the server generates and sends a 128-bit random number (i.e. the challenge) to this client. Then, the client takes the HMAC of this random number using the lower half of the hash of his/her password as the key (256-bit). Remember that we always use SHA3-512 as the hash algorithm on the password. Then, the client sends this HMAC to the server. The hash algorithm used in HMAC is also SHA3-512. If the server cannot verify the HMAC, it sends a negative acknowledgment message such as “Authentication Unsuccessful” to the client. If the server can verify the HMAC, the client is authenticated. In this case, the server sends a positive acknowledgment message such as “Authentication Successful”, together with the encrypted channel keys and IV as described below. Both positive and negative acknowledgments must be encrypted using AES-256 in CBC mode and then ciphertext must be signed by server's signature RSA key. As AES encryption key and IV, you will use the hash of the reversed password that was received during enrollment phase. The lower half (256-bit) of this hash is the AES-256 encryption key, and the 2nd quarter (128-bit, i.e. the lower half of the upper half) is the IV.

If the HMAC is verified successfully, the server must send the AES-256 symmetric encryption key, 128-bit IV and 256-bit HMAC key that are specific to the channel that the client subscribed to. To accomplish this, Server GUI should include a master secret part for each channel and the entered master secret will be used to generate these keys and IV for the corresponding channel. When "generate key" button is clicked for the specific channel, server takes the hash of the master secret using SHA3-512 and splits it to generate the required AES key and IV (lower 32 bytes for AES key, next 16 bytes for IV, 16 bytes not used). For the HMAC key of the channel, server takes the hash of the reversed master secret using SHA3-512 and use lower 32 bytes, and discards the rest.

Then, the server encrypts the channel keys and IV using AES-256 algorithm in CBC mode. The key and IV used for this encryption are derived from the hash of the reversed password (same as above). The server should append the encrypted keys to “Authentication Successful” message and sign the concatenated message using the signing RSA key of the server. Server, finally, sends (Authentication Successful || encrypted keys and IV) message and its signature to the client.

When the client receives this message, first of all, it verifies that the received signature is valid. If the signature is not valid, it disregards the message. It decrypts the content using the key and IV derived from the hash of the reversed password (using the same splitting logic as in the server side) and loads the keys and IV for secure channel communication. After decryption, client checks whether the response is "error" or "success". If it is a failed authentication, the client may try again.

As you can deduce from the flow above, channel keys and IVs must have been generated before the client authentication. You should handle this case as well. One way of doing this is that during login, instead of sending channel keys and IV, a message saying "Channel Unavailable" can be sent to the client instead of successful/unsuccessful authentication message. In this way, the clients may try over and over again.

Another thing that you might already deduced is that the channel keys and IV must not be permanently stored at the Server side; they are generated when the Server opens up through the GUI (master secret entry and generate key button). When the Server GUI is closed, then these keys and IVs are forgotten. Moreover, once they are created, they should not change during the lifetime of Server GUI; you should handle this.

During the decryption of the server message, if the client enters a wrong password, decryption fails of course. You should catch this exception so that the client can enter another password after an error message.

Some Caveats

In the project, some random numbers will be generated. These random numbers must be cryptographically secure ones.

We will test various failed authentication and wrong password cases (such as wrong keys, modified messages, etc.) during the demos; so, please test your own codes accordingly.

So far, it should be clear that the client needs to load the server's RSA public keys from files provided. Moreover, the server needs to load its own RSA public/private key pairs from files. For these purposes, both client and server need to browse the file system to choose the key file(s).

Please remark that this is a client-server application, meaning that the server listens on a predefined port and accepts incoming client connections. The listening port number of the server must be entered via the server GUI. Clients connect to the server on the corresponding port and identify themselves with usernames. Server needs to keep the usernames of currently connected clients in order to avoid the same username to be connected more than once at a given time. There might be one or more different clients connected to the server at the same time. Each client knows the IP address and the listening port of the server (to be entered through the GUI).

Provided RSA Keys

Client has:

- *server_enc_dec_pub.pem*: This file includes Server's RSA-3072 public key for encryption operations in PEM format.
- *server_sign_verify_pub.pem*: This file includes Server's RSA-3072 public key for signature verification purposes in PEM format.

Server has:

- *server_enc_dec_pub_prv.pem*: This file includes Server's RSA-3072 public/private key pair for encryption and decryption operations. The keys are in PEM format.
- *server_signing_verification_pub_prv.pem*: This file includes Server's RSA-3072 public/private key pair for signing and verification operations. The keys are in PEM format.

Secure Message Broadcast and Disconnection

Secure broadcasting is basically sending encrypted and authenticated messages among the connected clients via the server using the channel keys distributed during user authentication.

After secure login (authentication), the authenticated clients will have received *channel keys* from the server for the channels they subscribed to. Then, clients will be able to send secure and authenticated messages to the Server. After receiving such a message, the Server will send it to all connected clients that are subscribed to the same channel. While doing so, the server will not decrypt/verify the encrypted messages, it will only relay them to the subscribed clients, who will make the decryptions and message verifications. Since each client is subscribed to only one channel, he/she does not need to select a channel before sending.

When a client wants to broadcast a message, it should encrypt its message using 256-bit AES algorithm in CBC mode using the channel encryption key and IV. Then it should take the HMAC of this encrypted message using the channel HMAC key. The hash algorithm used in HMAC is also SHA3-512. User concatenates encrypted message and its HMAC together and sends to the server.

After the Server receives this packet, it broadcasts it to all of the connected clients that are subscribed to the same channel as the sending client (including the sending client). Here please remark that the Server does not decrypt and verify HMAC; it broadcasts unchanged.

When a client receives a broadcast message from the server, it decrypts the message using channel key and checks the validity of the HMAC. If everything is all right, the decrypted message is displayed in the GUI; otherwise, the message is discarded, and error prompt is displayed in the GUI.

Since there are three different channels that clients can subscribe to, on the Server GUI we should see three text widgets to easily follow the sent/received messages.

A client may disconnect from the server by pressing a disconnect button or by closing the window. After disconnection, the same user may want to login again by running a brand-new authentication phase; but he/she will not enroll again.

If the Server disconnects by pressing a disconnect button or by closing the window, the connected clients must understand this and get disconnected as well. Later, when the Server opens again, the clients can make fresh connections using previously completed enrollments for existing users.

The Client and Server applications must not crash while handling disconnections.